

Question 1 – Comparison of MinorCPU and O3CPU

		Minor CPU	O3 CPU
Clock Cycles	cpu.riscv	28.024.976	13.014.009
	mem.riscv	50.953.122	46.202.969
Instructions	cpu.riscv	25.002.864	
	mem.riscv	24.427.862	
IPC	cpu.riscv	0,892164	1,921227
	mem.riscv	0,479418	0,528708
L1\$ Miss Rate	cpu.riscv	0,000094	0,000231
	mem.riscv	0,000026	0,000054
L1D \$ Miss Rate	cpu.riscv	0,000007	0,000042
	mem.riscv	0,173884	0,181418
L2\$ Miss Rate	cpu.riscv	0,984000	0,995495
	mem.riscv	0,002535	0,004380

Question 1.1

When switching from MinorCPU to O3CPU the reduction of execution cycles for each program is:

$$Drop_{cpu} = \frac{28.024.976 - 13.014.009}{28.024.976} \approx 0,53 \text{ ή } \mathbf{53\%}$$

$$Drop_{mem} = \frac{50.953.122 - 46.202.969}{50.953.122} \approx 0,09 \text{ ή } \mathbf{9\%}$$

So the compute-intensive program has a much greater reduction.

Question 1.2

The speedup of O3CPU compared to MinorCPU for each program is:

$$Speedup_{cpu} = \frac{28.024.976}{13.014.009} \approx 2,15$$

$$Speedup_{mem} = \frac{50.953.122}{46.202.969} \approx 1,1$$

Question 1.3/4

The compute-intensive program benefits more from the O3CPU than the memory-intensive one. We would expect the opposite since during the time a cache miss is resolved, the next instructions can continue to execute in the pipeline and thus mask this delay. In this particular case this is not true because we do not have enough arithmetic instructions between load/stores and so we end up with stalls due to structural hazards (we have load/store instructions to execute but we don't have enough resources to execute them on).

Question 1.5

The number of instructions executed in each program for both MinorCPU and O3CPU is exactly the same. This is expected because in both cases we execute the same compiled file which has a predefined number of instructions. The CPU model on which we run it only affects the performance of the execution.

Question 2 – Effect of the cache hierarchy

		Minor CPU				O3 CPU			
		A	B	C	D	A	B	C	D
Clock Cycles	cpu.riscv	28024976	28024976	28025133	28024976	13014010	13014010	13014073	13014009
	mem.riscv	57869255	244377799	53512336	50950518	52707686	236705224	48639688	46200418
Instructions	cpu.riscv	25002864							
	mem.riscv	24427862							
IPC	cpu.riscv	0,892164	0,892164	0,892159	0,892164	1,921227	1,921227	1,921217	1,921227
	mem.riscv	0,422122	0,099959	0,456490	0,479443	0,463459	0,103200	0,502221	0,528737
L1\$ Miss Rate	cpu.riscv	0,000094	0,000094	0,000104	0,000094	0,000232	0,000232	0,000241	0,000231
	mem.riscv	0,000026	0,000026	0,000028	0,000026	0,000054	0,000054	0,000056	0,000054
L1D \$ Miss Rate	cpu.riscv	0,000007				0,000042			
	mem.riscv	0,186232	0,186231	0,192524	0,173884	0,188372	0,189338	0,199456	0,181481
L2\$ Miss Rate	cpu.riscv	0,984000	0,984000	0,907407	0,984000	0,991031	0,991031	0,956710	0,995495
	mem.riscv	0,025079	0,799045	0,002275	0,002520	0,026768	0,800853	0,003931	0,004354

Cache Configuration	L1I (KiB)	L1D (KiB)	L2
A	16	16	256 KiB
B	16	16	64 KiB
C	8	8	1 MiB
D	32	32	1 MiB

Question 2.1

To determine which program is most affected by changes in cache sizes, we will calculate the standard deviation of clock cycles for each program and for each CPU (i.e., the colored cells of the table for both CPUs). So, we have:

Program	CPU	
	Minor	O3
cpu.riscv	67,98299	27,427176
mem.riscv	82.425.189	81.232.946

As can be seen, the memory-intensive program is affected by orders of magnitude more than the compute-intensive one. This makes sense since its locality is very bad and with different cache sizes (e.g. as the capacity increases) it has more room to fetch data and instructions that may be used in the future.

Question 2.2

The compute-intensive program is not affected as much by changes in cache sizes, since it has much higher data locality. Instruction locality does not affect the comparison, since the L1 miss rate is practically constant for both programs. The fact that it has higher data locality, in this case, means that even if the cache size increases, the program cannot utilize it (even its smallest configuration is enough).

Question 2.3

As far as the compute-intensive program is concerned, all configurations give us similar values for miss rates. Therefore, we expect the clock cycles to be similar. This is true as we showed in question (2.1).

As far as the memory-intensive program is concerned, macroscopically, all configurations have similar miss rate values for L1. The same applies to L2 except for configuration B where the miss rate is much higher. In this case the clock cycles are significantly affected. More specifically for the other configurations, C and D are quite close and achieve better performance than A. This is mainly due to the miss rate of L2 since of A is one order of magnitude larger. We also see that D is better than C. This is due to the miss rate of L1D where C is 0.02 higher.

Question 3 – Effect of ROB on O3CPU

		O3 CPU			
		Default	ROB A	ROB B	ROB C
Clock Cycles	cpu.riscv	13.014.009	11.139.079	9.763.527	13.014.051
	mem.riscv	46.202.969	46.932.614	46.451.025	46.217.797
Instructions	cpu.riscv	25.002.864			
	mem.riscv	24.427.862			
IPC	cpu.riscv	1,921227	2,244608	2,560843	1,921221
	mem.riscv	0,528708	0,520488	0,525884	0,528538
"Lost" cycles (idle cycles)	cpu.riscv	9.111	9.055	8.597	9.137
	mem.riscv	8.985	8.971	9.116	9.332
Full ROB Events	cpu.riscv	4	3.375.199	3.000.055	1.000.096
	mem.riscv	66.683	13.233.175	13.138.642	147.047
Full Register Events	cpu.riscv	0			
	mem.riscv	605	0		

ROB config	Default	A	B	C
Entries	192	32	48	96

Question 3.1

Lost cycles:

According to [this](#), idleCycles counts the clock cycles that the processor has not scheduled a command to execute (no program is actively running e.g. waiting for IO or sleep() has been called). This is different from cycles where the pipeline is stalled.

Full ROB Events:

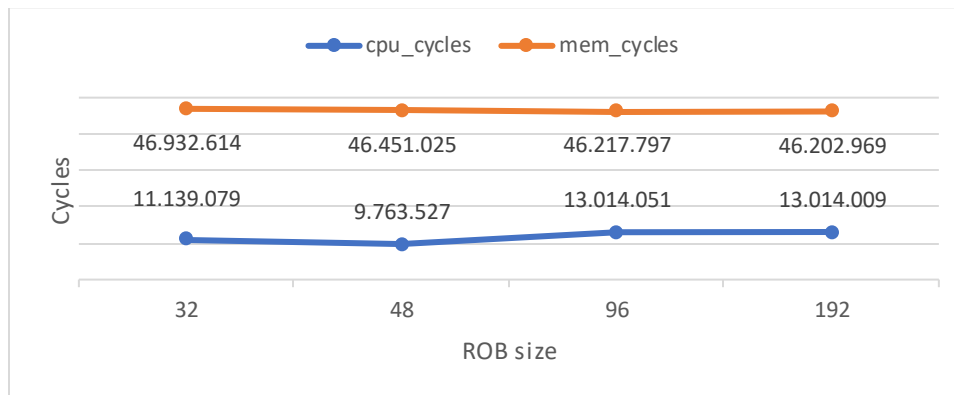
It counts the number of times the Reorder Buffer was filled with records. This is important when we study its possible sizes, because when such an event occurs it means that the rename stage cannot process incoming instructions (ROB does not have an empty space to hold the in-order order of the instruction) and we have a stall of the pipeline front-end. Because of this, instructions that could potentially be executed cannot be passed to the issue queue.

Full Register Events:

It counts the number of times the Physical Register File was filled with records. This statistic is the same as Full ROB Events but is triggered when the destination register of the incoming instruction tries to rename itself to one of the physical registers, but they are all occupied.

Question 3.2

The relationship of execution cycles to the size of ROB is shown in the following diagram:



The slope of the trendlines (difference in cycles per extra entry in the ROB) for each program is:

$$\text{cpu} : \sim 16.500 \quad \text{mem} : \sim -3.500$$

The memory-intensive program benefits most from increasing the size of the ROB.

Question 3.3

What we would expect for both programs is that as we increase the size of the ROB, the execution cycles will decrease since we would have fewer Full ROB Events and therefore fewer stalls. We would even expect the compute-intensive program to benefit more since the instructions that remain in the ROB need a relatively short execution time (they do not make as many memory accesses) and thus their scheduling can be done much more efficiently. On the other hand, for the memory-intensive program, instructions that remain in the ROB need a long execution time (memory accesses) and thus do not leave as many empty entries for new instructions that could be put in the issue queue.

What we expect is confirmed for the memory-intensive program but not for the compute-intensive one. In the second we see the reduction we expect when we go from 32 to 48 entries (a decrease of 12%, while in the first we have 1%) but not from 96 entries and above.

Question 4 – Study of physical integer registers

		O3 CPU			
		Default	PRF A	PRF B	PRF C
Clock Cycles	cpu.riscv	13.014.009	11.514.126	13.014.009	13.014.009
	mem.riscv	46.202.969	46.726.127	46.208.861	46.202.969
Instructions	cpu.riscv	25.002.864			
	mem.riscv	24.427.862			
IPC	cpu.riscv	1,921227	2,171495	1,921227	1,921227
	mem.riscv	0,528708	0,522788	0,528640	0,528708
"Lost" cycles (idle cycles)	cpu.riscv	9.111	9.153	9.111	9.111
	mem.riscv	8.985	9.561	9.775	8.985
Full ROB Events	cpu.riscv	4	0	4	4
	mem.riscv	66.685	0	5	66.683
Full Register Events	cpu.riscv	0	7.625.139	0	0
	mem.riscv	0	42.578.845	609.825	605

PRF config	Default	A	B	C
Entries	256	64	128	192

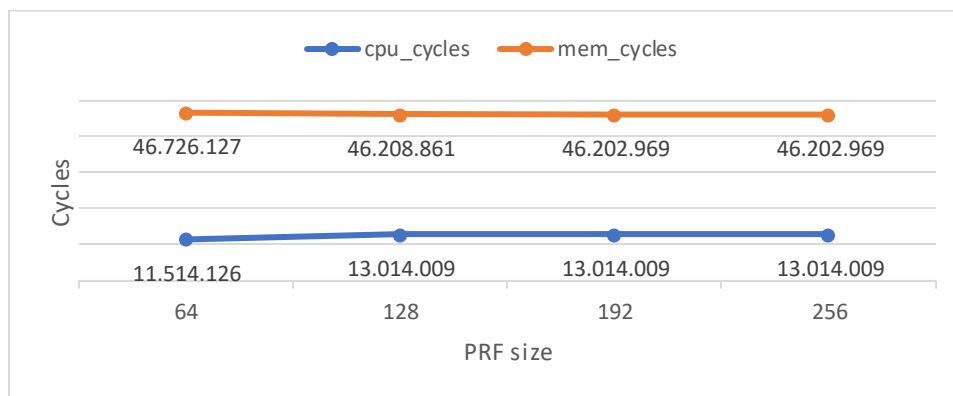
Question 4.1

The behavior we expect is similar to what we expected from question 3.3 since ROB and PRF are working together in the rename stage.

To see which program is most affected by changes in PRF sizes, we will calculate the standard deviation of clock cycles for each program. So, we have:

Program	Standard Deviation
cpu	649.468,39
mem	225.696,44

The compute-intensive program appears to be significantly affected compared to the memory-intensive one, but they are not affected in the same way as the size of the PRF increases. From this diagram:



We see that as we increase the size of the PRF, the cycles of the compute-intensive program decrease while the cycles of the memory-intensive program increase. Also, for $PRF_{size} \geq 192$ we see that the clock cycles are the same. This is because the size of the ROB is 192 and so: when a command reaches the rename stage, it needs a PRF entry (for renaming) and one for ROB (to maintain the correct commit order of the instructions and recovery in case of branch misprediction). When $PRF_{size} \geq 192$ the PRF may have empty slots for the instruction that came in, but the ROB is full, so the renaming cannot take place. Overall, what we expect is fully confirmed for the memory-intensive program but for the compute-intensive one we see an increase in cycles.

Question 5 – Final conclusion

The main conclusion we draw from the previous questions is that there is no ideal microarchitecture for all executable programs and that all the choices we make are based on trade-offs that we confirm experimentally. We also see that even changes in the design of a single unit of the microarchitecture, can greatly affect the orchestration with all the other units.

Question 6

In the memory-intensive program, the fact that the memory allocated by malloc() at the beginning is not freed up at the end of the execution is not good practice, but it is not a mistake because the operating system undertakes to free it after the execution ends. In the event that our program had malloc() in a loop and did not manage memory properly, we would have a memory leak, and this would lead to a continuous increase in memory until the program was terminated by the operating system or finished running.